Chapter 10: Printing and Print Preview

This chapter explains how the printing process works by exploring the relationship between screen output and printer output, and the similarities and differences of these two output options. It also covers how you can add print support to your application by using MFC AppWizard. The later topics of this chapter cover obtaining printer information and using this information to control the printed document.

Objectives

After completing this chapter, you will be able to:

- Describe the printing process and the default printing capabilities provided by MFC for an MFC AppWizard-generated application.
- Add default printer support to your application.
- Retrieve information relating to printers and print jobs at run time.
- Enhance default printer support to implement custom requirements.

306 Adding Default Printer Support

Adding default printer support to your application is a simple task when you use MFC AppWizard.

This section describes the printing process and the printer support that MFC provides to implement printing. It also explains how you can enable default printer support for your applications.

The last topic outlines some printing-related issues you must handle in your application.

This section includes the following topics:

- Introduction to the Printing Process
- Enabling Default Printer Support
- MFC Support for Printing
- Screen Display vs. Printing

Notes

307 Introduction to the Printing Process

The printing process involves three phases:

1. The request phase

   In this phase, the application receives a print request from the user.

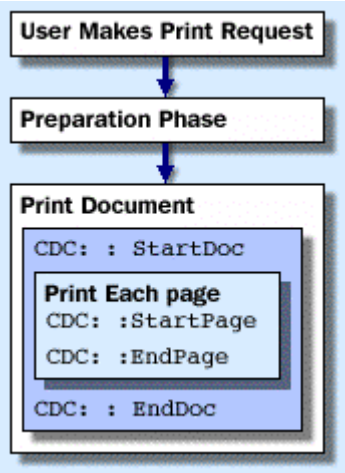
2. The preparation phase

   In this phase, the application displays the **Print** dialog box, in which the user can specify the settings for the print job.

3. The printing phase

   In this phase, the application divides the document into a set of pages based on user settings. The application then prints each page along with its header and footer.

The following illustration shows the three phases of the printing process.



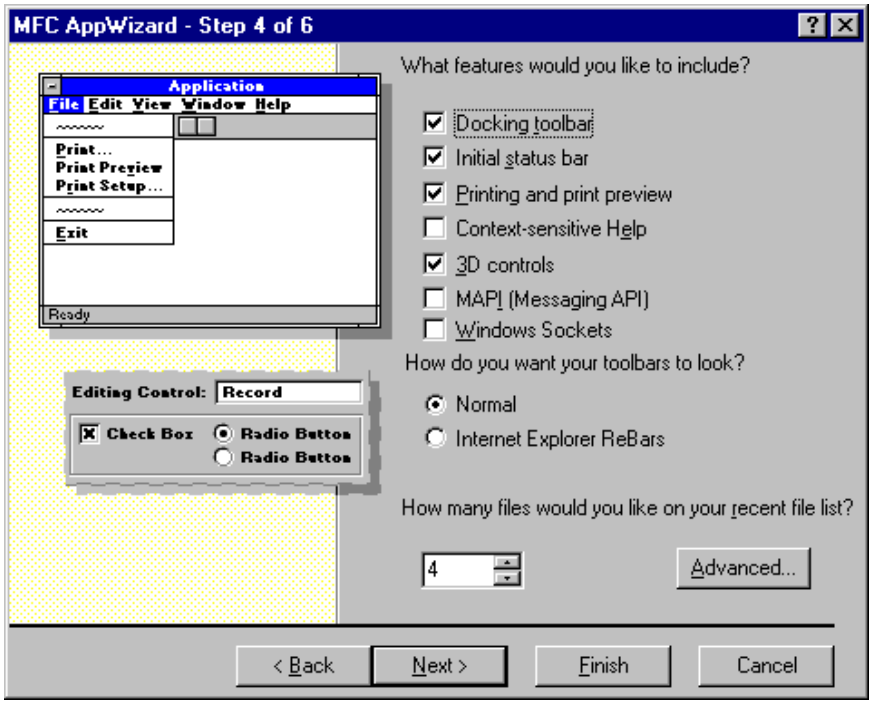
Notes

308 Enabling Default Printer Support

When you use MFC AppWizard to create an application, the wizard offers you the option of adding default printer support. To add this support to your application, select the **Printing and Print Preview** check box in Step 4 of MFC AppWizard.

To see an illustration of the **MFC AppWizard - Step 4** dialog box with the printer support option enabled, click this icon.

(CD-ROM displays the following illustration.)



When you select this option, MFC adds the following commands to the **File** menu of your application:

- **Print**
- **Print Preview**
- **Print Setup**

MFC also adds the code to implement these commands in the implementation file of the view class.

Notes

309

MFC Support for Printing

MFC provides printer support in the following ways:

- By adding **CView** member functions to your view class
- By implementing device-independent display of data

CView Member Functions for Printing

When you enable printing for an application in MFC AppWizard, MFC automatically adds the required **CView** member functions to the view class of the application. The framework calls these functions during the three phases of printing (request, preparation, and printing) to implement a print job.

The following table lists the **CView** functions that are called for printing a document.

Name	Related printing phase	Result
OnPreparePrinting	Request	Inserts default values in the **Print** dialog box and displays it; also creates the device context for the printer.
OnBeginPrinting	Preparation	Allocates fonts or other Graphic Device Interface (GDI) resources; also sets the length of the document that needs to be printed.
OnPrepareDC	Preparation	Adjusts the attributes of the device context for a given page; also performs print-time pagination.
OnPrint	Printing	Prints each page, including header and footer.
OnEndPrinting	Printing	Releases GDI resources or reestablishes default settings.

Device-Independent Display

In MFC-based applications, the **OnDraw** member function of the **CView** class handles the drawing operation, regardless of whether the device is the screen or a printer. MFC implements this feature by passing a pointer to a **CDC** object as a parameter, as shown in this example:

```
virtual void OnDraw(CDC* pDC);
```

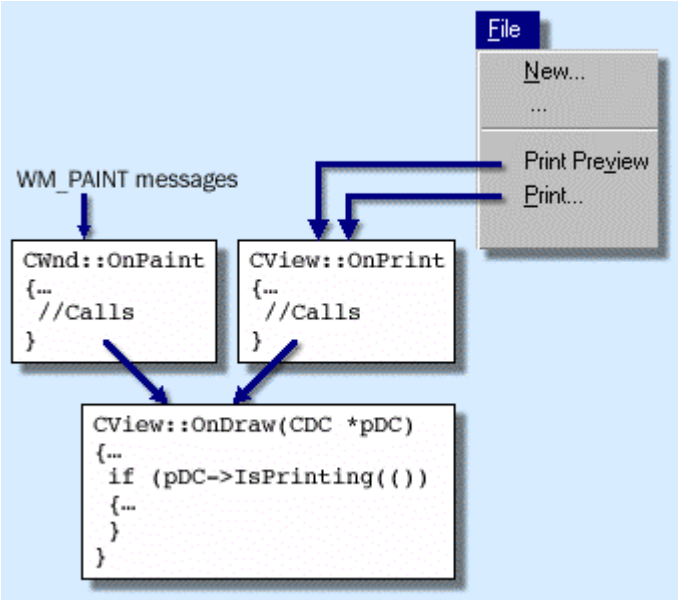
The **CDC** object represents the device context that should receive the image produced by **OnDraw**.

When the window displaying the document receives a WM_PAINT message, the framework calls **OnDraw** and passes it a device context for the screen. Accordingly, the **OnDraw** function sends its output to the screen.

When the window displaying the document receives a print command from the user, the framework calls the **OnPrint** function. The **OnPrint** function in turn calls the **OnDraw** function with a device context for a printer. The **OnDraw** function then sends its output to the printer.

Notes

The following illustration shows how MFC-based applications can perform simple printing without requiring extra effort on your part.



Notes

310 Screen Display vs. Printing

While there are many similarities between printing and screen display, there are also some significant differences between the two that your application must anticipate and handle. For example, when you print a document from your application, the application must divide the document into distinct pages and print them one at a time, rather than display whatever portion is visible in a window as with screen output.

When you develop your application, you must consider some characteristics of printed pages, including:

- Size of the paper used for printing

  The user may use standard letter-size paper, an envelope, or some other size paper.
- Page orientation

  The user may choose landscape or portrait orientation for the print job.

The MFC Library cannot predict how your application will handle these types of issues, so it provides a protocol for you to add these capabilities. The protocol is a set of overrideable **CView** class member functions that return information on print settings selected by the user.

Notes

311 Enhancing Printer Support

You can enhance printer support by overriding the **CView** member functions. For example, if the printed version of your document needs headers and footers, you may need to override the **OnPrint** function to add code to include headers and footers in the printout.

To see an animation that shows how you can enhance printer support by overriding **CView** member functions, click this icon.

 (CD-ROM plays the animation, "Overriding CView-Derived Functions.")

To enhance printer support, you need information related to the following:

- Settings of the default printer
- Settings chosen by the user for the current print job
- Attributes of the printer device context

This section begins by explaining how you can retrieve information related to printing and then describes how you can use this information in the **CView** functions.

This section includes the following topics:

- Determining Printer Characteristics
- Retrieving Information About the Current Print Job
- Retrieving Information About the Printer Device Context
- Overriding CView-Derived Functions

Notes

312 Determining Printer Characteristics

You can obtain information about default printer settings in two ways:

- On application startup

  If your application needs information about the characteristics of the Windows default printer on startup, override the **InitInstance** member function of the **CWinApp**-derived class.

  To see sample code that queries for printer information at application initialization, click this icon.  (CD-ROM displays the following sample code.)

```
// Getting printer information on application startup
// Samples\CH10\Prprevue
// Prprevue.cpp
BOOL CMyApp::InitInstance()
{
    ...

    // These 2 statements are needed to give the app the
    // default printer information. Without them, the call
    // to CreatePrinterDC fails.
    PRINTDLG dlg;
    GetPrinterDeviceDefaults(&dlg);

    CDC dc;
    if (0 != CreatePrinterDC(dc))
    {
        TEXTMETRIC tm;
        dc.GetTextMetrics(&tm);
        int x = dc.GetDeviceCaps(HORZRES);
        int y = dc.GetDeviceCaps(VERTRES);
        ...
    }

    ...
}
```

- Subsequent to application startup

  The default printer setting may be changed either by the user or by another application while your application is running. To track changes to the default printer, implement the **OnWinIniChange** member function of the **CWnd**-derived class. The framework calls this function to allow your application to handle a Windows message.

Notes

313 Retrieving Information About the Current Print Job

You use the pInfo parameter that is passed to the **CView**-derived functions to obtain information about the current print job. The pInfo parameter of the **CView::*OnXxxx*** functions is a pointer to an object named **CPrintInfo**. The **CPrintInfo** object stores information about the print job.

CPrintInfo Class Members

The following table describes some **CPrintInfo** class members.

Data member	Description
m_pPD	Contains a pointer to the **CPrintDialog** object that is used for the **Print** dialog box.
m_bDirect	Contains a flag indicating whether the document is being printed directly (without displaying the **Print** dialog box).
m_bContinuePrinting	Contains a flag indicating whether the framework should continue the print loop.
m_nCurPage	Identifies the number of the page that is currently being printed.
SetMaxPage	Sets the number of the last page of the document.

 (CD-ROM displays the following tip.)

For a complete list of **CPrintInfo** class members, search for “CPrintInfo" in MSDN Library Visual Studio 6.0.

Notes

Using the CPrintDialog Object

The **CPrintDialog** object is a member of the **CPrintInfo** class and encapsulates the services provided by the Windows common dialog box for printing. For example, by using **CPrintDialog**, you can determine the print range and the name of the selected print driver. The **CPrintDialog** object is created by using the **CPrintInfo** constructor **m_pPD**. The following table describes some **CPrintDialog** class members.

Data member	Description
m_pd	A structure used to customize a **CPrintDialog** object. A member of this structure, **Flags**, specifies the dialog box initialization flags. For example, PD_DISABLEPRINTTOFILE disables the **Print to File** check box, and PD_ALLPAGES indicates that the **All** option button was selected when the user closed the dialog box.
GetDefaults	Stores the current printer defaults without displaying a dialog box.
GetCopies	Retrieves the number of copies requested.
GetDeviceName	Retrieves the name of the currently selected printer device.

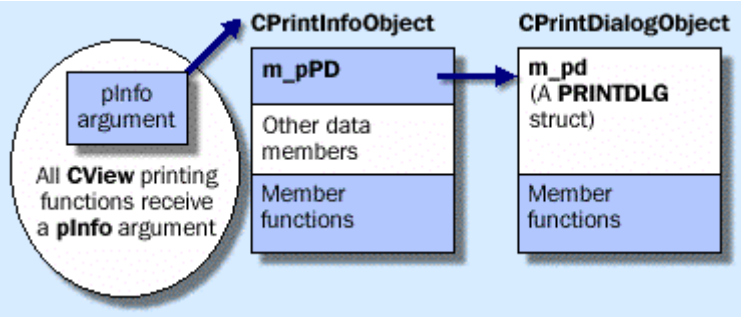
(CD-ROM displays the following tip.)

For a complete list of the **CPrintDialog** class members, search for “CPrintDialog" in MSDN Library Visual Studio 6.0.

The following example code shows how to access a member of the **CPrintDialog** object through the pointer to the **CPrintInfo** object:

```
pInfo->m_pPD->m_pd.Flags |= PD_DISABLEPRINTTOFILE;
```

To see an illustration that shows how the pInfo argument is used to access the **CPrintInfo** and **CPrintDialog** objects, and shows the relationships between these two objects, click this icon.
(CD-ROM displays the following illustration.)



Notes

314 Retrieving Information About the Printer Device Context

The pDC parameter that is passed to all **CView::*OnXxxx*** functions, except **OnPreparePrinting**, points to the device context object on which the image will be rendered. Use the member functions of the **CDC** class to obtain information related to the device context.

 (CD-ROM displays the following tip.)

For more information about the printer device context, search for “CDC Class Members” in MSDN Library Visual Studio 6.0.

Getting Information on Display Resolution

Use the **GetDeviceCaps** member function to get information about the display resolution. The following example code shows how you can retrieve the resolution settings as numeric values:

```
int x = dc.GetDeviceCaps(HORZRES);
int y = dc.GetDeviceCaps(VERTRES);
```

For more information about this function, search for "GetDeviceCaps" in MSDN Library Visual Studio 6.0.

Sending Escape Codes to the Printer

You can call the **Escape** member function of the pDC parameter from the **OnPrepareDC** member function to send escape codes to the printer on a page-by-page basis.

Getting the Current Status of a Device Context

You use the **IsPrinting** member function of the pDC parameter to determine whether the device context is used for printing.

Notes

315 Overriding CView-Derived Functions

This topic lists some overridable **CView**-derived functions and describes the reasons for overriding them. It also gives an example of each of the overridden functions.

The code samples discussed in this topic appear in the sample application Prprevue, which is located in the folder \Samples\Ch10.

OnPreparePrinting

In the **OnPreparePrinting** function, you can customize the **Print** dialog box before it is displayed to the user. To see sample code that disables the **Print to File** check box and displays the **Print** dialog box to the user, click this icon.

 (CD-ROM displays the following sample code.)

```
// Customizing and Displaying the Print dialog box
// Samples\CH10\Prprevue
// Prinview.cpp
BOOL CPrintView::OnPreparePrinting(CPrintInfo* pInfo)
{
    // In this function, we can modify the behavior of the printer
    // dialog box before it's displayed. It's best to determine the
    // number of pages in the print job AFTER the dialog box has been
    // displayed.
    pInfo->m_pPD->m_pd.Flags |= PD_DISABLEPRINTTOFILE;

    return DoPreparePrinting(pInfo);
}
```

Notes

OnBeginPrinting

In the **OnBeginPrinting** function, you can examine changes made to the **Print** dialog box. To see sample code that determines how many lines to print on each page based on the orientation selected by the user, click this icon.

 (CD-ROM displays the following sample code.)

```
// Determining the number of pages to be printed based on user's
// selection
// Samples\CH10\Prprevue
// Prinview.cpp
void CPrintView::OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo)
{
    TEXTMETRIC tm;
    pDC->GetTextMetrics(&tm);
    int y = pDC->GetDeviceCaps(VERTRES);

    // Allow for 3 lines at the top and 2 at the bottom
    int lines = (y / tm.tmHeight) - 5;

    // It's possible that the user has changed the printer
    // such that the number of lines it supports is less than
    // what the user currently wants. If so, both the desired
    // lines per page (stored in the view object) and the maximum
    // number of lines the current printer mode supports
    // (stored in the app object) will have to be changed.
    if (lines < GetPageLength())
    {
        SetPageLength(lines);
        CPrintApp * cpa = (CPrintApp *)::AfxGetApp();
        cpa->SetMaxPageLength(lines);
    }

    // The max page needs to be rounded up if the number of
    // lines to be printed isn't exactly divisible by
    // the lines per page.
    CPrintDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    int pagecount = pDoc->LinesOnDocument() / GetPageLength();
    if ( 0 != (pDoc->LinesOnDocument() % GetPageLength()) )
        pagecount++;

    pInfo->SetMaxPage(pagecount);

    CScrollView::OnBeginPrinting(pDC, pInfo);
}
```

The previous example uses the **SetMaxPage** function to establish how many pages will be processed by the printing loop.

Notes

OnPrepareDC

You can override the **OnPrepareDC** function to set per page properties. To see sample code that positions the pages based on page number, click this icon.

 (CD-ROM displays the following sample code.)

```
// Setting page properties
// Samples\CH10\Prprevue
// Prinview.cpp
void CPrintView::OnPrepareDC(CDC* pDC, CPrintInfo* pInfo)
{
    CScrollView::OnPrepareDC(pDC, pInfo);

    // On a scroll view, if you want to modify the DC, do it AFTER
    // the call to the base class version.
    // Just for demo purposes, this program prints odd-numbered
    // pages shifted 500 printer units to the right.
    if (pDC->IsPrinting())
        if (0 == (pInfo->m_nCurPage & 1))
            pDC->SetViewportOrg(0, 0);
        else
            pDC->SetViewportOrg(500, 0);
}
```

In the previous example, if the page number is even, the printed page is shifted to the left. If the page number is odd, the printed page is shifted to the right.

Notes

OnPrint

The actual printing of each page occurs in the **OnPrint** function. To see sample code that prints a header, the page content, and the footer for each page, click this icon.

 (CD-ROM displays the following sample code.)

```
// Handling the actual printing
// Samples\CH10\Prprevue
// PrinView.cpp
void CPrintView::OnPrint(CDC* pDC, CPrintInfo* pInfo)
{
    int BeginningRow, EndingRow, i, x, y;
    TEXTMETRIC tm;
    CSize cs;
    CString s;

    CPrintDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    pDC->GetTextMetrics(&tm);

    // From the page currently being printed, determine the
    // starting row of this simulated text document.
    BeginningRow = (pInfo->m_nCurPage - 1) * GetPageLength();

    // From that determine the ending row, which can't be
    // greater than the number of lines in the document.
    EndingRow = BeginningRow + GetPageLength();
    if (EndingRow > pDoc->LinesOnDocument())
        EndingRow = pDoc->LinesOnDocument();

    // Load the page header from the string table, then print it
    // centered at the top of the page.
    s.LoadString(IDS_COMPANY_LOGO);
    cs = pDC->GetTextExtent(s);
    x = pDC->GetDeviceCaps(HORZRES);
    pDC->TextOut((x - cs.cx) / 2, 0, s);

    // Print the requested number of lines on the current page.
    y = 3 * tm.tmHeight;
    for (i = BeginningRow; i < EndingRow; i++, y += tm.tmHeight)
        pDC->TextOut(0, y, pDoc->GetLine(i + 1));
```

(Code continues on following page.)

Notes

```
    // Print the page footer, centered on the bottom line.
    s.Format(IDS_PAGE, pInfo->m_nCurPage);
    cs = pDC->GetTextExtent(s);
    y = pDC->GetDeviceCaps(VERTRES);
    pDC->TextOut((x - cs.cx) / 2, y - cs.cy, s);

    // Since this isn't a WYSIWYG application, the base class's function
    // should not be called, since it would then call OnDraw, which in
    // this application is intended ONLY for screen display.
    //CScrollView::OnPrint(pDC, pInfo);
}
```

OnEndPrintPreview

You can alter the behavior of Print Preview in a number of ways, such as:

- Display the Print Preview window with a scroll bar so that the user can view other pages in the document.
- Cause Print Preview to maintain the user's position in the document by beginning its display at the current page.
- Alter the appearance of information that is displayed in Print Preview, such as page numbers.

Notes

To see sample code that overrides the **OnEndPrintPreview** function to cause the document view to display the last page displayed by Print Preview, click this icon.

 (CD-ROM displays the following sample code.)

```
// Setting the current page of the document view based on the page
// previewed last
// Samples\CH10\Prprevue
// Prinview.cpp
void CPrintView::OnEndPrintPreview(CDC* pDC, CPrintInfo* pInfo,
                                   POINT point, CPreviewView* pView)
{
    // We want to be able to scroll in the view to the page corresponding
    // to where the user scrolled during print preview.

    // The pDC argument points to a printer DC. We need a DC for the view.
    CClientDC dc(this);

    // We'll need to know a character's height.
    TEXTMETRIC tm;
    dc.GetTextMetrics(&tm);

    CPoint pt;
    // Simple multiplication determines the point.
    pt.y = (pInfo->m_nCurPage - 1) * tm.tmHeight * GetPageLength();
    pt.x = 0;
    ScrollToPosition(pt);

    CScrollView::OnEndPrintPreview(pDC, pInfo, point, pView);
}
```

Notes

318 Sample Applications

Here is the description of the sample application related to this chapter. The sample application is located in the folder \Samples\Ch10.

Sample application subfolder	Description of application
Prprevue	Implements default print and print preview functionality in an MFC application. Shows how to enhance printing capability by overriding **CView** printing functions.

Notes

320

Self-Check Questions

1. Which statement determines where to redirect the output of the OnDraw function?

- A. **CDC*** pDC
- B. **const CRect&** *rcBounds*
- C. **const CRect&** *rcInvalid*
- D. **CPrintInfo*** pInfo

2. If you do not know the number of pages to print at the start of the process, which function will you override to provide print-time pagination?

- A. **CView::OnPreparePrinting**
- B. **CView::DoPreparePrinting**
- C. **CView::OnPrepareDC**
- D. **CView::OnPrint**

3. What are the three major steps in the printing process?

- A. Displaying the **Print** dialog box, customizing it, and printing the document
- B. Customizing the **Print** dialog box, displaying it, and printing the document based on the settings selected by the user
- C. Showing a preview of the document, displaying the **Print** dialog box, and printing the document
- D. Overriding the **CView** member functions, displaying the **Print** dialog box, and printing the document.

Notes